

บทที่ 2

Extensible Markup Language (XML)

2.1 บทนำ

XML ย่อมาจาก Extensible Markup Language เป็นภาษาที่ใช้อธิบายข้อความในเอกสาร เพื่อให้ข้อมูลนั้นมีโครงสร้าง ภาษา XML นั้นไม่ได้เน้นที่การแสดงผล หรือควบคุมการทำงานแต่ ภาษา XML เน้นที่การรองรับการแลกเปลี่ยนข้อมูลระหว่างกัน เป็นอีกภาษาหนึ่งที่เกิดขึ้นมาเพื่อรองรับความก้าวหน้าทางด้านเทคโนโลยีเว็บในปัจจุบันโดยที่ผู้ใช้งานสามารถที่จะกำหนดและสร้างแท็กขึ้นมาใช้ได้เอง โดยมีการประกาศโครงสร้างของเอกสารขึ้นมาก่อนซึ่งมีด้วยกัน 2 ชนิด คือ DTD และ XML Schema

XML เป็นภาษาที่เป็น Markup อีกภาษาหนึ่งซึ่งเกิดในปี 1996 โดย World Wide Web Consortium (W3C) โดยมีการตั้งกลุ่มผู้ทำงาน XML (XML Working Group หรือ XWG) เพื่อทำการสร้าง XML ขึ้นมา

ข้อมูลที่น่าสนใจอยู่บน Web Page มีลักษณะที่เป็น Semistructure คือมีลักษณะเป็นข้อมูลคล้ายข้อมูลแบบ Structure แต่มีบางส่วนที่ไม่อยู่ในลักษณะของ Structure เช่น

- ไม่สามารถกำหนดอิลิเมนต์ที่ตายตัวให้เหมาะสมสำหรับทุกๆ หน้าของ Web page
- มีบางอิลิเมนต์ที่ปรากฏได้หลายครั้งหรือบางหน้าของ Web ก็ไม่ปรากฏเลย
- บางอิลิเมนต์ข้อมูลย่อยภายในมีค่าเพียงแค่นั้น (Atomic value) แต่บางอันก็มีหลายค่า (Structure values) อยู่ในรูปของ Structure ย่อยอีกที
- อิลิเมนต์ทั้งหมดในแต่ละหน้าของ Web page เมื่อมองโครงสร้างที่เกิดขึ้น แต่ละหน้าก็มีโครงสร้างที่ไม่เหมือนกัน

เพราะฉะนั้นถ้าหากนำเอาข้อมูลของ Web เก็บลงใน Relational Database หรือ Object - Oriented Database จะทำให้การออกแบบทำได้ยาก เพราะฉะนั้นจึงมีการสร้าง XML (eXtended - Markup Language) ซึ่งเป็นภาษาที่เหมาะสมในการจัดเก็บข้อมูลแบบ Semistructure ขึ้นมาเพื่อรองรับการรูปแบบเหล่านี้

2.2 DTD

หนทางที่เอกสาร XML จะถูกกำหนดโครงสร้างขึ้นมาได้นั้นคือการใช้ Document Type Definition(DTD) การที่เราจะสามารถระบุกฎเกณฑ์ที่ว่านี้ ขึ้นมาได้จะทำการแลกเปลี่ยน ประมวลผล และแสดงผล XML ในสภาพแวดล้อมกว้าง การใช้ DTD จะทำให้เราไม่เพียงแต่ล่วงรู้ได้ว่าเอกสาร XML ทำตามกฎไวยากรณ์ของข้อกำหนด XML หรือไม่ แต่ยังล่วงรู้ได้ว่ามันทำตามกฎของตัวเองเรากับเนื้อหาแล้ว โครงสร้างหรือไม่อีกด้วย

DTD ย่อมาจาก Document Type Definition เป็นภาษาที่ทำหน้าที่กำหนดกฎ กติกาให้กับเอกสาร XML เช่น ว่าต้องประกอบไปด้วย อิลิเมนต์ อะไรบ้าง แอตทริบิวต์ อะไรบ้าง เป็นต้น DTD ถือว่าเป็นส่วนเพิ่มเติม (optional) ของเอกสาร XML นั่นคือเราจะกำหนดให้เอกสาร XML ของเรามี DTD กำกับอยู่ด้วยหรือไม่ก็ได้จะกำหนดให้อยู่ภายในเอกสาร XML เลย หรือจะแยกเป็นอีกไฟล์ก็ได้ ถ้าเป็นแบบแยกเป็นอีกไฟล์ก็ได้จะได้อีกไฟล์ที่มีนามสกุลเป็น .dtd

```
<!ELEMENT Book (Title, Author, Year, ISBN)>
<!ELEMENT Title (#PCDATA)>
<!ELEMENT Author (#PCDATA)>
<!ELEMENT Year (#PCDATA)>
<!ELEMENT ISBN (#PCDATA)>
```

รูปที่ 2.1 ตัวอย่าง DTD

จากรูปที่ 2.1 เป็นตัวอย่างการเขียน DTD ซึ่งถ้าจะสร้างเอกสาร XML จะต้องมี อิลิเมนต์ที่ชื่อ Book โดยในอิลิเมนต์ Book ก็จะต้องประกอบไปด้วยอิลิเมนต์ลูกๆที่ชื่อว่า Title , Author , Year และ ISBN โดยที่แต่ละอิลิเมนต์ลูกๆจะเก็บข้อมูลเป็น #PCDATA ซึ่งก็คือเก็บข้อมูลเป็นชนิด text ซึ่งจากตัวอย่างเป็นการเก็บข้อมูลที่เกี่ยวข้องกับข้อมูลของหนังสือ

จากเอกสาร DTD ที่ได้กล่าวมาเราสามารถสร้างเอกสาร XML ได้ดังนี้

```
<?xml version="1.0" encoding="UTF-8"?>
<Book>
  <Title> XML Web Technologies </Title>
  <Author> Ander Moller </Author>
  <Year> 2006 </Year>
  <ISBN> 0-321-26966-7 </ISBN>
</Book>
```

รูปที่ 2.2 ตัวอย่างเอกสาร XML

จากรูปที่ 2.2 เป็นตัวอย่างเอกสาร XML ข้างบนอธิบายได้ดังนี้ เอกสาร XML เก็บข้อมูล Book ซึ่งภายในจะเป็นชื่อหนังสือที่ชื่อ XML Web Technologies ชื่อผู้แต่งคือ Ander Moller ปี 2006 และหมายเลข ISBN คือ 0-321-26966-7

2.2.1 การประกาศ DTD ในเอกสาร

การประกาศ DTD สามารถทำได้ 2 วิธี คือแบบ Internal และแบบ External ซึ่งการสร้างแบบ Internal จะทำภายในเอกสาร XML และจะนำส่วนของ DTD มาอยู่ในเครื่องหมาย [และ] ส่วนการสร้างแบบ External จะทำการสร้าง DTD ขึ้นมาแล้วได้ไฟล์เป็นนามสกุล .dtd และในเอกสาร XML จะมีการระบุว่าจะใช้รูปแบบโครงเอกสารจากที่ไว้ด้วย

- **Internal DTD**

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE Book [
    <!ELEMENT Book (Title, Author, Year, ISBN)>
    <!ELEMENT Title (#PCDATA)>
    <!ELEMENT Author (#PCDATA)>
    <!ELEMENT Year (#PCDATA)>
    <!ELEMENT ISBN (#PCDATA)>
]>
<Book>
    <Title> XML Web Technologies </Title>
    <Author> Ander Moller </Author>
    <Year> 2006 </Year>
    <ISBN> 0-321-26966-7 </ISBN>
</Book>
```

รูปที่ 2.3 ตัวอย่าง Internal DTD

- **External DTD**

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE Book SYSTEM "Book.dtd">
<Book>
    <Title> XML Web Technologies </Title>
    <Author> Ander Moller </Author>
    <Year> 2006 </Year>
    <ISBN> 0-321-26966-7 </ISBN>
</Book>
```

รูปที่ 2.4 ตัวอย่าง External DTD

2.2.2 การสร้าง DTD

การสร้าง DTD จะแบ่งออกเป็น 3 หัวข้อหลักๆ คือ การประกาศค่าอิลิเมนต์ (Element), การประกาศค่าแอตทริบิวต์ (Attributes) และการประกาศค่าเอนทิตี (Entity)

2.2.2.1 การประกาศค่าอิลิเมนต์ (Element)

“อิลิเมนต์” หรือที่บางคนเรียกกันว่า “แท็ก” จะมีรูปแบบการประกาศดังนี้

```
<!ELEMENT ชื่ออิลิเมนต์ (ข้อมูลภายในอิลิเมนต์)>
```

รูปที่ 2.5 รูปแบบการประกาศค่าอิลิเมนต์

ข้อมูลภายในอิลิเมนต์หนึ่งๆ อาจเป็นไปได้ 3 อย่างคือ เป็นอิลิเมนต์อื่นๆ, เป็นข้อความปกติ และอาจไม่มีอะไรอยู่เลยหรือที่เรียกว่าอิลิเมนต์ว่าง (Empty Element)

ตัวอย่างแบบต่างๆดังนี้

- แบบอิลิเมนต์อื่นๆอยู่ภายใน

```
<Book>
    <Title> XML Web Technologies </Title>
    <Author> Ander Moller </Author>
</Book>
```

รูปที่ 2.6 ตัวอย่างอิลิเมนต์แบบมีอิลิเมนต์อยู่ภายใน

- แบบเป็นข้อความอยู่ภายใน

```
<Title> XML Web Technologies </Title>
```

รูปที่ 2.7 ตัวอย่างอิลิเมนต์แบบมีข้อความอยู่ภายใน

- แบบไม่มีอะไรอยู่เลย

```
<Title/>
```

รูปที่ 2.8 ตัวอย่างอิลิเมนต์แบบไม่มีอะไรอยู่ภายใน

- แบบที่มีอิลิเมนต์ปนอยู่กับข้อความ (Mixed content)

```
<Book> Programing
    <Title> XML Web Technologies </Title>
</Book>
```

รูปที่ 2.9 ตัวอย่างอิลิเมนต์แบบผสมอยู่ภายใน

ใน XML จะมีเครื่องหมายพิเศษ 3 แบบคือ +, * และ ? ซึ่งใช้เพื่อแสดงว่าอิลิเมนต์นั้นจะสามารถปรากฏได้กี่ครั้ง โดยแต่ละเครื่องหมายจะมีคุณลักษณะดังนี้

- + หมายถึง จะต้องมียิลิเมนต์นั้นตั้งแต่ 1 อิลิเมนต์ขึ้นไป (คือ 1, 2, 3)
 - * หมายถึง จะไม่มีอิลิเมนต์นั้นก็ได้ หรือมีตั้งแต่ 1 อิลิเมนต์ขึ้นไปก็ได้ (คือ 0, 1, 2, 3)
 - ? หมายถึง จะไม่มีอิลิเมนต์นั้นก็ได้ หรือมีอิลิเมนต์ก็ได้แต่มีได้แค่ 1 ตัว (คือ 0, 1)
- ส่วนที่ไม่มีเครื่องหมายอะไรเลยนั้นหมายถึงอิลิเมนต์นั้นจะต้องมีอิลิเมนต์ปรากฏ 1 ตัว

2.2.2.2 การประกาศค่าแอตทริบิวต์ (Attributes)

แอตทริบิวต์เป็นตัวขยายข้อมูลให้กับ Element อีกทีหนึ่ง

```
<ATTLIST ชื่ออิลิเมนต์ ชื่อแอตทริบิวต์ ชนิดข้อมูลของแอตทริบิวต์
    (#REQUIRED | #IMPLIED | #FIXED) ค่าปกติของแอตทริบิวต์>
```

รูปที่ 2.10 รูปแบบการประกาศค่าแอตทริบิวต์

ชนิดข้อมูลของ แอตทริบิวต์ (Attributes)

Type	Description
CDATA	เป็นตัวอักษรธรรมดาที่ไม่ประกอบด้วย ตัวอักษรที่เป็น Markup
ENTITIES	ชื่อเอนทิตีหลายตัว(ซึ่งต้อง declare ใน DTD) แยกกันด้วยช่องว่าง
ENTITY	(whitespace)
Enumerated	ชื่อเอนทิตี (ซึ่งต้อง declare ใน DTD)
ID	แทน list of values (ต้องใช้ค่าใดค่าหนึ่งจาก list)
IDREF	เป็นคุณสมบัติของ XML ที่ชื่อจะต้องมีซ้ำกัน นั่นคือแต่ละ attribute ที่มีค่า ID จะมีชื่อที่ไม่ซ้ำกัน
IDREFS	จะถือค่าของ ID attribute ของบางอิลิเมนต์โดยปกติอิลิเมนต์ที่เป็นอิลิเมนต์ปัจจุบัน(current element)จะเกี่ยวข้อง
NMTOKEN	แสดง ID ของอิลิเมนต์หลายค่าซึ่งแยกโดยช่องว่าง (whitespace)
NMTOKENS	แสดงชื่อ XML ที่เกี่ยวข้อง
NOTATION	แสดงชื่อของ XML ที่เกี่ยวข้องกันหลายค่าในลิสต์ ซึ่งแยกกันโดยช่องว่าง (whitespace)
	แสดงชื่อของ notation ซึ่งต้อง declare ใน DTD

ตารางที่ 2.1 แสดงชนิดข้อมูลของแอตทริบิวต์

2.2.2.3 การประกาศค่าเอนทิตี (Entity)

```
<!ENTITY ชื่อเอนทิตี ทรัพยากร>
```

รูปที่ 2.11 รูปแบบการประกาศค่าเอนทิตี

เอนทิตี มีอยู่ 2 ชนิด คือ general entity และ parameter entity โดย general entity จะอยู่ใน content ของเอกสาร XML ซึ่งจะขึ้นต้นด้วยเครื่องหมาย & และลงท้ายด้วยเครื่องหมาย ; ส่วน parameter entity จะใช้เอกสาร DTD ซึ่งจะเริ่มต้นด้วยเครื่องหมาย % และลงท้ายด้วยเครื่องหมาย ;

2.2.3 DTD Comments

การเขียนคอมเมนต์ในส่วนของ DTD จะมีลักษณะเหมือนกับการเขียนคอมเมนต์ใน XML นั่นคือข้อความที่จะคอมเมนต์จะต้องอยู่ระหว่างเครื่องหมาย `<!--` และ `-->`

```
<!--ข้อความคอมเมนต์ -->
```

รูปที่ 2.12 รูปแบบการใช้คอมเมนต์

2.3 XML Schema

XML Schema เกิดขึ้นมาเพราะ DTD ไม่ได้เขียนอยู่ในรูปแบบเหมือน XML จึงทำให้เหมือนต้องศึกษาอีกภาษาหนึ่งในการจัดการ XML ซึ่ง XML Schema มีลักษณะของตัวอักษรคล้ายๆ กับภาษา XML และภาษา XSL ทำให้ไม่เกิดความสับสนมากมากในการศึกษาและการทำงาน ซึ่ง XML Schema ทำหน้าที่เป็นภาษาสำหรับกำหนดความถูกต้องของข้อมูล เป็นวิธีที่ใหม่กว่า DTD แต่การเขียน XML Schema จะมีความยากกว่า DTD

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<xs:schema      xmlns:xs="http://www.w3.org/2001/XMLSchema"
  elementFormDefault="qualified">
  <xs:import
    namespace="http://www.w3.org/XML/1998/namespace"/>
  <xs:element name="Book">
    <xs:complexType>
      <xs:sequence>
        <xs:element ref="Title"/>
        <xs:element ref="Author"/>
        <xs:element ref="Year"/>
        <xs:element ref="ISBN"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
  <xs:element name="Title" type="xs:string"/>
  <xs:element name="Author" type="xs:string"/>
  <xs:element name="Year" type="xs:string"/>
  <xs:element name="ISBN" type="xs:string"/>
</xs:schema>
```

รูปที่ 2.13 ตัวอย่าง XML Schema

จากรูปที่ 2.13 เป็นตัวอย่าง XML Schema ที่แปลงมาจาก DTD ที่ได้เคยยกตัวอย่างตามรูปที่ 2.1 มาแล้ว จะเห็นได้ว่า XML Schema นั้นมีความยุ่งยากกว่าจะอาจจะยากกว่า DTD แต่ XML Schema นั้นก็มีข้อดีบางอย่างที่ DTD ไม่มี เช่น การทำ Namespaces ซึ่ง DTD นั้นไม่สามารถทำได้

2.3.1 Namespaces

Namespaces เป็นการใช้เพื่อสำหรับลดความสับสนในเอกสาร XML ซึ่งภายในเอกสาร XML นั้นอาจจะมีอีลิเมนต์เหมือนกัน แต่จริงๆแล้วอาจจะสื่อความหมายที่ต่างกัน อยู่ภายใต้อีลิเมนต์ที่ต่างกัน แล้วอาจจะถูกเรียกใช้ภายในเอกสาร XML ไฟล์เดียวกัน การประกาศ prefix และ Namespaces จะอาศัย แอตทริบิวต์ xmlns (ย่อมาจาก XML namespace) ตามรูปแบบดังนี้

```
<xmlns:prefix= "ชื่อNamespaces" >
```

รูปที่ 2.14 รูปแบบการใช้ Namespaces

2.3.2 Simple Type

Simple Type คือ อีลิเมนต์ที่มีข้อมูลภายในเป็นสตริง หรือตัวเลข หรือข้อมูลพื้นฐานอื่นๆ มีรูปแบบดังนี้

```
<element name="ชื่ออีลิเมนต์" type="ชนิดข้อมูล">
```

รูปที่ 2.15 รูปแบบการใช้ Simple Type

2.3.3 Complex Type

Complex Type คืออีลิเมนต์ที่บรรจุอีลิเมนต์อื่นเอาไว้ภายในด้วย หรือเป็นอีลิเมนต์ที่มีแอตทริบิวต์ประกอบอยู่ด้วย ดังตัวอย่างต่อไปนี้

```
<xs:complexType name="Book">
  <xs:sequence>
    <xs:element name="Title" type="xs:string"/>
    <xs:element name="Author" type="xs:string"/>
    <xs:element name="Year" type="xs:string"/>
    <xs:element name="ISBN" type="xs:string"/>
  </xs:sequence>
</xs:complexType>
```

รูปที่ 2.16 ตัวอย่างการใช้ Complex Type

จากตัวอย่างรูปที่ 2.16 เป็นการแสดงตัวอย่างการใช้ Complex Type โดยมีอิลิเมนต์ที่ชื่อว่า Book โดยภายในอิลิเมนต์ Book จะเก็บอิลิเมนต์ Title, Author, Year และ ISBN ไว้ภายใน โดยแต่ละอิลิเมนต์ย่อยจะเก็บข้อมูลแบบ string

2.4 Well – Formed

ในการสร้างเอกสาร XML ให้เป็นเอกสารที่ Well – Formed นั้นมีข้อกำหนดมากมายเกี่ยวกับโครงสร้างของเอกสาร XML ที่จะทำให้ตัวเอกสารนั้น Well – Formed ต้องปฏิบัติตามกฎข้อบังคับเหล่านี้

- เอกสารต้องมี Element ระดับบนสุดเพียง Element เดียวเท่านั้น (ซึ่งเรียกว่า Document Element หรือ Root Element) โดย Element อื่น ๆ ทั้งหมดจะต้องซ่อนอยู่ภายใน
- Element ต้องซ่อนอยู่อย่างถูกต้อง นั่นหมายความว่าถ้ามี Element ซ่อนอยู่ภายใน Element อื่นๆ ต้องปิด Element นั้นภายใน Element เดียวกับที่ Element นั้นซ่อนอยู่
- แต่ละ Element ต้องมีแท็กเริ่มต้นและแท็กปิดท้าย ซึ่งต่างจาก HTML ตรงที่ XML ไม่สามารถให้คุณละเว้นแท็กปิดท้ายได้ เพราะเมื่อแท็กไม่ครบคู่จะทำให้เบราว์เซอร์ไม่เข้าใจคำสั่งของ Element นั้นได้ (ในบทที่ 3 คุณจะได้เรียนรู้วิธีใช้ Empty Element หรือ Element ว่างที่ไม่มีข้อมูลอยู่ภายใน)
- ชื่อของ Element ในแท็กเริ่มต้นจะต้องมีชื่อที่ตรงกับชื่อของแท็กปิดท้ายเท่านั้น
- ชื่อของ Element มีลักษณะเป็น Case-sensitive (การแยกระหว่างตัวอักษรตัวใหญ่และตัวเล็ก) ซึ่งอันที่จริงข้อความทั้งหมดภายใน XML Markup จะต้องมีลักษณะเป็น Case-sensitive

2.5 Case-Sensitivity

สิ่งที่สำคัญอีกสิ่งหนึ่งที่คำนึงอยู่เสมอคือแท็กใน XML นั้นจะให้ความสำคัญของตัวอักษรพิมพ์เล็ก-ใหญ่ (ซึ่งแตกต่างเป็นอย่างมากกับ HTML ซึ่งไม่ให้ความสำคัญกับเรื่องนี้) นั่นหมายความว่า <Book> แตกต่างจาก <BOOK> แล้วแตกต่างจาก <book> อีกด้วย

เรื่องแบบนี้บางทีอาจกลายเป็นเรื่องเล็กน้อยสำหรับผู้ใช้ XML ที่ใช้ภาษาอังกฤษเป็นหลัก เนื่องจากคำต่างๆ ในภาษาอังกฤษสามารถแปลงจากตัวพิมพ์เล็กไปเป็นพิมพ์ใหญ่ได้ โดยไม่เสียความหมายของคำไป แต่ในภาษาอื่นๆ เกือบทั่วโลก เรื่องของความสำเร็จตัวพิมพ์เล็ก-ใหญ่สามารถใช้ได้หรือไม่สำคัญมากมายนัก

2.6 Valid

เอกสาร XML ที่ผ่านการ Well – Formed อย่างเดียวนั้นยังไม่เพียงพอ เอกสารตัวนั้นจะต้องถูกต้อง (Valid) ด้วย โดยการจะถูกต้องใช้ได้หรือไม่นั้นก็ขึ้นอยู่กับเอกสารดังกล่าวทำตามกรอบของ DTD (Document Type Definition) หรือ XML Schema ที่กำหนดไว้หรือไม่

2.7 ข้อดีของ XML

XML เป็นภาษาที่ยืดหยุ่นต่อข้อมูลการเก็บข้อมูลที่มีลักษณะเป็น Semistructure และนอกจากนั้น XML สามารถเป็นตัวกลางในการแปลงข้อมูลที่อยู่ในลักษณะการเก็บที่ต่างกัน เช่น Relational Database, Object Oriented Database หรือ Text file ทำให้สามารถเข้าถึงข้อมูลได้เหมือนข้อมูลทั้งหมดอยู่ในที่เดียวกัน และมีรูปแบบเดียวกัน ซึ่ง XML มีข้อดีต่าง ๆ ดังนี้

- XML มีความเป็นสากล เพราะสนับสนุนการใช้งาน Unicode
- XML มีลักษณะที่เป็นโครงสร้าง ทำให้สามารถทำการตรวจสอบความถูกต้องของเอกสารทำได้ง่าย โดยใช้ DTD (Document Type Definition) หรือ Schema
- มีส่วนประกอบเพิ่มเติมที่ช่วยทำให้การสร้างเอกสาร XML มีประสิทธิภาพมากยิ่งขึ้น คือ Xpart XPointer XLink
- XML สามารถจัดเก็บและนำเสนอข้อมูลได้ทุกรูปแบบอย่างมีประสิทธิภาพ
- XML มีความยืดหยุ่นรองรับการเก็บข้อมูลที่มีการเปลี่ยนแปลงบ่อยหรือมีรูปแบบที่ไม่แน่นอน
- XML สามารถนำมาใช้งานได้ง่าย มีความซับซ้อนกว่า HTML เพียงเล็กน้อย และบราวเซอร์ก็เริ่มมีการสนับสนุนการใช้ XML
- XML มีรูปแบบที่เป็นมาตรฐาน